Egg Segmentation and Tracking: SAM vs. Classical Image Processing

Video tracking output (Task 4): Google Drive – tracked.mp4

Summary

task	what was done	headline result
1. Pretrained segmentation	Grounding DINO ("egg.") → boxes → SAM ViT-H → masks for all 110 images; visual QC, 3 images fixed with better prompts	138 eggs in 110 images (1–3 per image)
2. Classical segmentation	Leakage-aware 4:1 split by scene groups (DINOv2 embeddings + clustering); 27 logged trials leading to a threshold-sweep + watershed + egg-likeness-score algorithm	dev mIoU 0.704 (from 0.341 baseline)
3. Evaluation	IoU, Dice, precision/recall, boundary F1, instance F1, count accuracy, bootstrap CIs; per-scene, difficulty and failure-mode analysis	eval mIoU 0.539 [0.36, 0.70]; IoU 0.91 when the egg is found; failures driven by low egg-background contrast
4. Video tracking	Adapted segmenter + Kalman/Hungarian tracker with re-identification by elimination	2 IDs for 2 eggs, 0 identity errors in 12 checked key frames, 0 false positives on title cards

Repository layout: `notebooks/` (Task 1, Colab GPU), `src/eggseg/` (classical segmenter, metrics, tracker), `scripts/` (split, experiments, analysis, video), `configs/` (frozen parameters), `experiments/log.csv` (all trials), `outputs/` (masks, figures, video), `reports/` (this report). Setup and run instructions: `README.md`.

Task 1 – Segmentation with a pretrained model

Notebook: `notebooks/01_task1_sam_groundtruth.ipynb` (Google Colab, T4 GPU, ~3 s per image).

Method: Grounded-SAM

SAM (Segment Anything) is a *promptable*, class-agnostic segmenter: a ViT-H image encoder runs once per image and a light mask decoder turns a prompt (box or points) into a mask. SAM does not know what an "egg" is, so it is paired with **Grounding DINO**, an open-vocabulary detector that turns the text prompt `"egg."` into boxes:

```
image → Grounding DINO ("egg.", box threshold 0.30, text threshold 0.25) → box filtering → SAM ViT-H (one mask per box) → clean-up
```

- **Box filtering:** drop boxes covering > 90% of the image (bowl, table); non-maximum suppression at IoU 0.5 (duplicate boxes); drop "group boxes" that contain ≥ 2 other egg boxes (a box around a cluster of eggs).
- **SAM:** one box prompt per egg, `multimask_output=False` (a box is an unambiguous prompt).
- **Clean-up:** keep the largest connected component and fill holes (specular highlights can punch holes); overlaps between eggs resolved by painting larger masks first.
- **Outputs:** binary masks (ground truth for Tasks 2–3), instance masks (egg IDs), overlays, `detections.json` (boxes, scores, SAM predicted IoU, settings).

Visual quality control

All 110 overlays were inspected. 107 were correct as produced, including eggs partly hidden by leaves, a paper flap or a rod, eggs in water and on white tapioca pearls. Three were wrong and were fixed **with better SAM prompts, not by hand-painting**, so the ground truth stays model-generated and reproducible (overrides are logged in `detections.json`):

image	problem in the first run	fix
nd22, nd39	2nd box (score 0.36 / 0.49) = the egg's shadow on a printed lid	keep only the top-scoring box
nd117	lower box (0.38) = an empty egg-shaped tray cup; a rod splits the real egg, SAM masked one half and the largest-component clean-up dropped the rest	keep top box, add positive points on both halves and a negative point on the rod, keep all large components

All three false boxes scored below 0.5, and the two lowest-scoring boxes of the whole dataset (0.36, 0.38) were false: detector confidence is a useful pointer to images that need checking, but not a safe automatic filter (several true eggs scored 0.41–0.49).

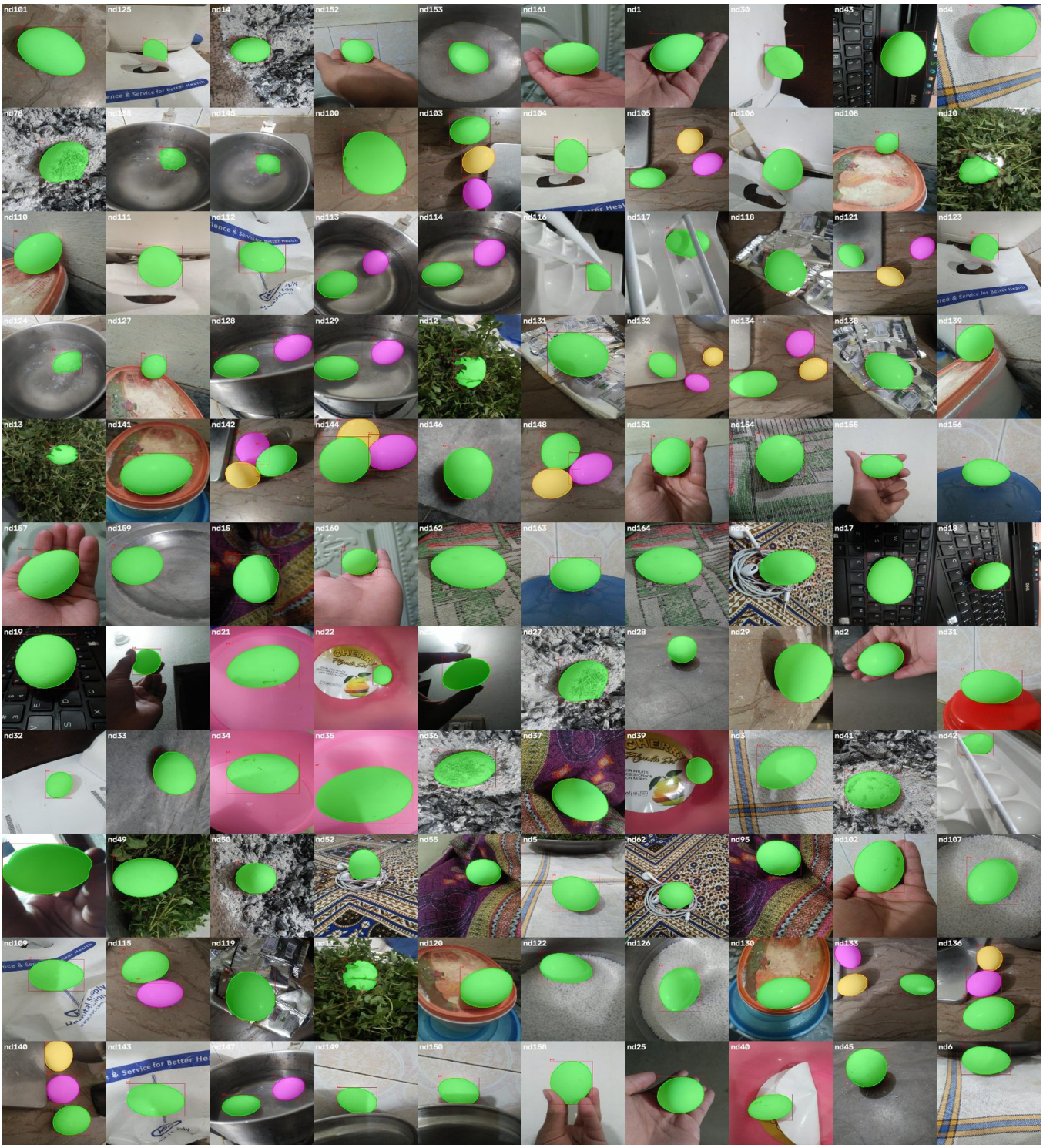


Task 1 QC: first run (left) vs. corrected prompts (right)

Result

138 eggs in 110 images: 93 images with 1 egg, 6 with 2, 11 with 3. SAM's own predicted IoU for its masks: median 0.985, minimum 0.905. Median egg area: 12% of the image.

Caveat: these masks are *pseudo* ground truth. They are very good but not perfect (e.g. boundaries under fingers or leaves), which caps the agreement measurable in Task 3.



All 110 SAM masks (overlays used for visual QC)

Task 2 – Segmentation with classical image processing

2.1 Leakage-aware dev / eval split (4:1)

The dataset has no augmented copies (all 110 source names unique), but many photos show **the same scene** (same table, bowl, hand or eggs). If near-duplicates land on both sides, tuning on dev "sees" eval and the eval score is optimistic. The split is therefore made by **scene group**, not by image (`scripts/make_split.py`):

1. **DINOv2 embedding** per image (computed in the Task 1 notebook); cosine similarity = scene similarity.
2. **Agglomerative clustering** (average linkage) at similarity 0.70. Thresholds 0.60–0.85 were compared: 0.85 leaves related scenes apart (leakage), 0.60 produces one 50-image group (4:1 impossible).
3. **Manual review** of every group: 5 same-scene pairs that the clustering missed were merged (documented in the script). Result: 25 scene groups.
4. **Whole groups assigned to eval**: among 5000 random assignments, choose the one with exactly 22 eval images, similar multi-egg share and the most eval groups.

	random image split	group split (used)
mean similarity of an eval image to its closest dev image	0.85	**0.65**
eval images with a near-twin (> 0.9) in dev	28%	**0%**

Result: 88 dev / 22 eval images, 9 scene groups in eval (egg tray, printed lid, white sheet, dark-room hand, steel bowl with 2 eggs, ...). All development used dev only; eval was run once with the frozen algorithm.



Scene groups (one row per group, eval groups marked)

2.2 Data analysis before designing the algorithm

Using the SAM masks on dev, each colour channel was scored by ROC-AUC = probability that a random egg pixel has a higher value than a random background pixel (`scripts/channel_analysis.py`):

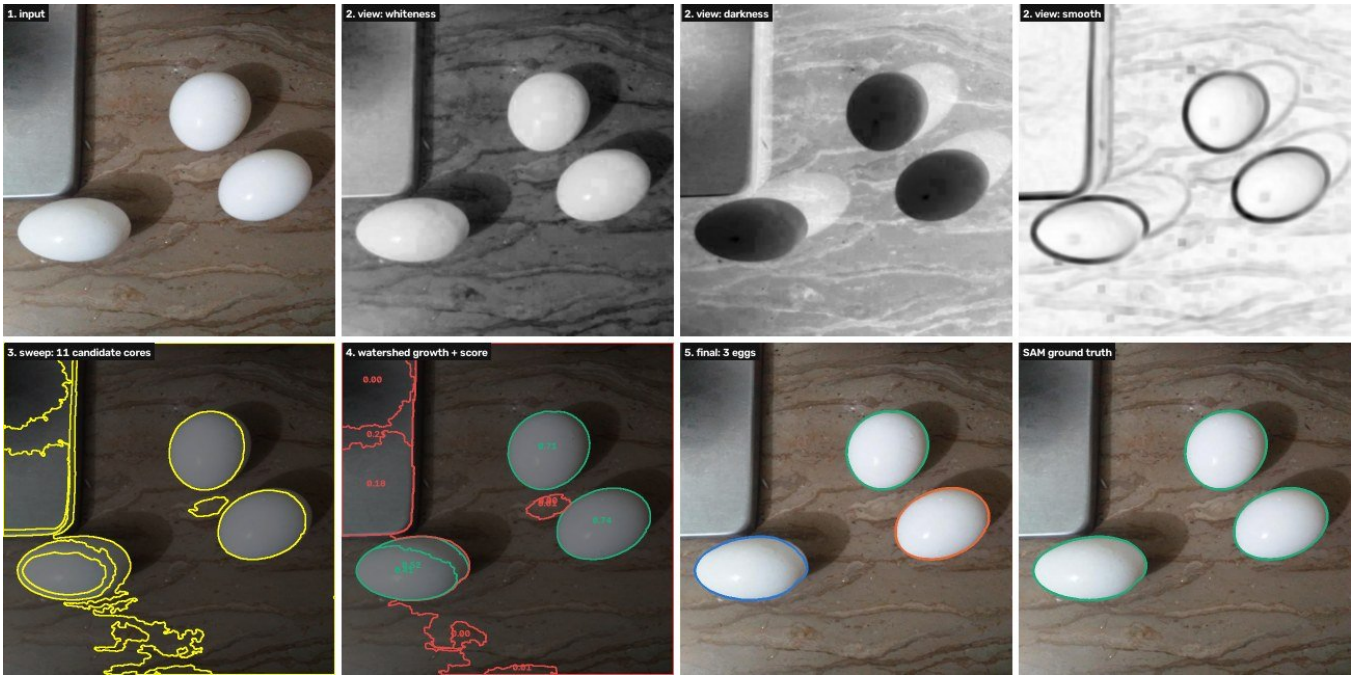
channel	per-image AUC (median)	global AUC	egg is
B, R, gray, L, V	0.885–0.890	0.83–0.85	brighter
S (saturation)	0.819	0.77	less saturated
Lab b, chroma	0.77–0.82	0.65–0.71	less colourful
H, Lab a	0.71–0.75	0.50–0.51	–

Conclusions: (1) eggs are "the bright, colourless thing", (2) no channel separates them alone, so shape knowledge is needed, (3) per-image thresholds beat one global threshold (0.89 vs 0.85), (4) the worst images (pink tub, white sheet, steel, foil, ash) predicted the later failure modes.

2.3 Final algorithm (`src/eggseg/classical.py`, parameters `configs/final_classical.json`)

1. **Three feature maps ("views")**, each blurred (Gaussian 5×5): *whiteness* = Lab L − 4·chroma (bright and colourless); *darkness* = inverted whiteness (an egg on white paper is slightly greyer); *smoothness* = inverted local standard deviation (15×15) – the shell is untextured.

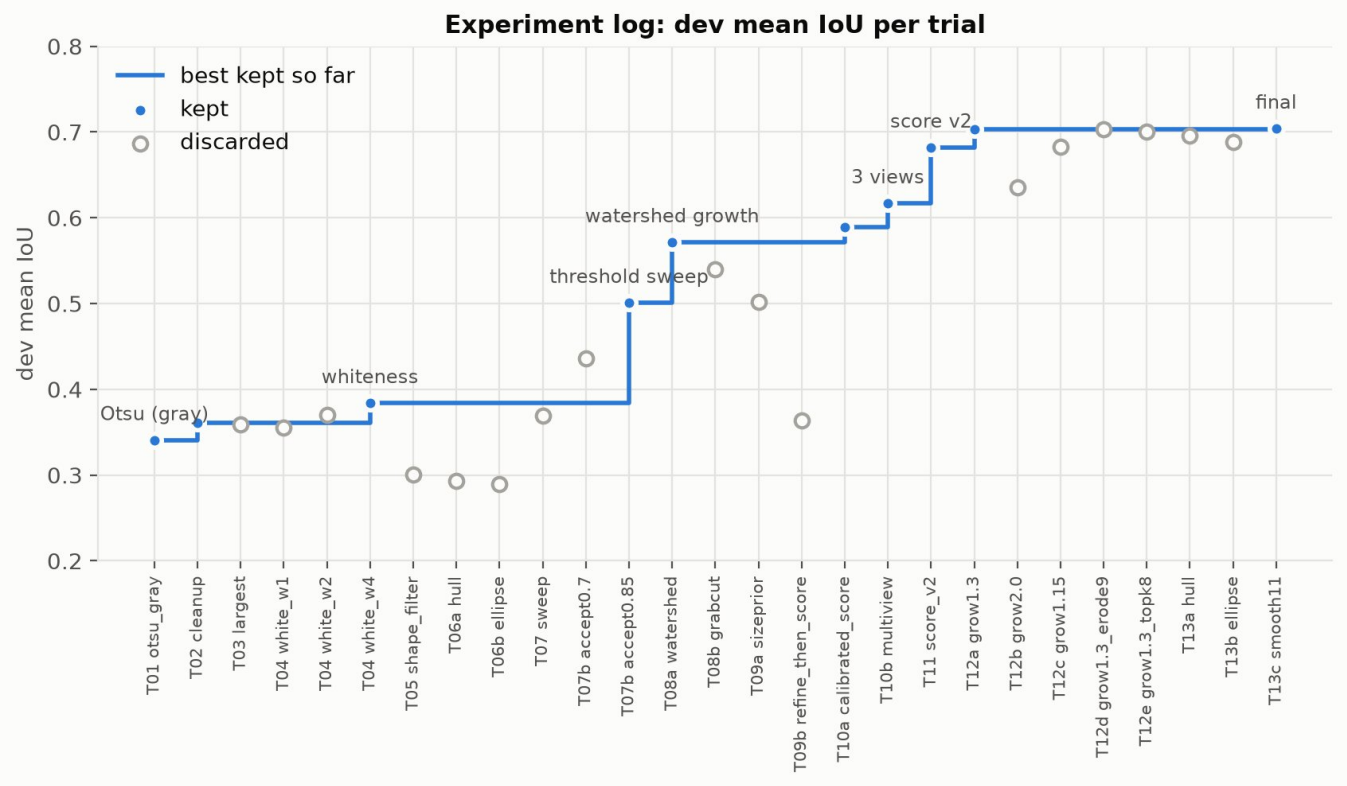
2. **Threshold sweep:** each view is thresholded at 24 levels between its 5th and 99.5th percentile; after a 5×5 opening, every connected blob (0.3%–60% of the image) at every level is a candidate. Candidates are scored by ellipse-fit IoU × solidity × contrast to a 7-px ring, and the top 5 distinct ones per view are kept (mask NMS).
3. **Watershed growth:** each candidate core is grown by **marker-controlled watershed** on the Lab colour gradient. Markers: the eroded core (egg) and everything outside 1.3 × the core's fitted ellipse (background). The egg interior is smooth, so the flood passes the shadowed side and stops at the real outline.
4. **Egg-likeness score** of each grown region (product of terms in [0, 1]): ellipse-fit IoU⁴ × solidity × size prior (log-normal, fit on dev eggs) × edge strength (outline gradient vs. a ring just outside) × sigmoid(brightness contrast to surroundings) × 0.5 if touching the image border.
5. **Selection:** best region always kept; further eggs if score ≥ 0.25 and overlap ≤ 20%. With several eggs, a **joint watershed** from all chosen cores splits touching eggs along their contact line.
6. **Smoothing:** opening + closing (11 px) of each egg mask.



Algorithm walkthrough on a dev image: views, 11 candidate cores, grown regions with scores (green = accepted, red = rejected), final result vs. SAM

2.4 How the experiments led to the algorithm

Every trial was run with `scripts/run_experiment.py`, which scores all 88 dev images against SAM and logs parameters and metrics to `experiments/log.csv` (each trial is reproducible from its logged parameters).



Dev mean IoU per trial (filled = kept, hollow = discarded)

The path in short:

- **T01–T04 (0.34 → 0.38):** a single global threshold on brightness or "whiteness" always merges the egg with bright background (hand, carpet, paper).
- **T05–T06 (failures, 0.29–0.30):** shape filters made it *worse* – they rejected the merged egg+background blob and the egg with it. Lesson: the problem is **how candidates are generated**, not how they are filtered.
- **T07 (0.50):** threshold sweep – at some level each egg separates as its own ellipse.
- **T08 (0.57):** watershed growth recovers the shadowed half (GrabCut tried, worse and slower).
- **T09 (failures):** a diagnostic (best achievable IoU among candidates vs. the chosen one) showed a saturated edge term instead of tuning blindly.
- **T10–T11 (0.59 → 0.68):** a table of all refined candidate regions with their IoU was used to design the score offline (AUC of each cue); three feature views raised the best achievable IoU from 0.68 to 0.75.
- **T12–T13 (0.70):** search-region size and smoothing; hull/ellipse completion rejected (over-fills occluded eggs). Further tuning gave < 0.002 – the algorithm was frozen to avoid over-fitting dev.

Experiment log (dev split, 88 images)

trial	what changed (parameters)	why	dev mIoU	Dice	inst. F1	decision
T01 otsu gray	Baseline: grayscale, per-image Otsu threshold, connected components	Simplest classical method; eggs are mostly the bright object	0.341	0.455	0.007	Baseline. Precision 0.37: hands, carpets, metal are bright too
T02 cleanup	+ Gaussian blur 5, opening 5, closing 9, fill holes, drop blobs < 0.2% of image	Remove speckles, fill glare holes	0.361	0.478	0.203	**Kept** (+0.02)
T03 largest	+ keep only the largest blob	Most images contain one egg	0.359	0.461	0.231	Discarded: drops real eggs in multi-egg images (instance recall 0.37 to 0.21)
T04 white w1	Feature 'whiteness' = Lab L - 1 x chroma	Channel analysis: eggs are bright AND colourless	0.355	0.472	0.191	Discarded: w=4 better
T04 white w2	Whiteness with chroma weight 2	Weight sweep	0.370	0.484	0.206	Discarded: w=4 better
T04 white w4	Whiteness with chroma weight 4	Weight sweep	0.384	0.496	0.205	**Kept**, but one global threshold still merges egg and background
T05 shape filter	+ drop blobs with solidity < 0.85 or ellipse-fit IoU < 0.75	Eggs are convex ellipses	0.301	0.360	0.352	Discarded: egg is merged with background into one non-elliptic blob and removed with it (recall 0.92 to 0.54). Insight: the problem is candidate generation
T06a hull	+ replace blob by convex hull	Recover the shadowed side of the egg	0.293	0.354	0.343	Discarded: same merged-blob problem
T06b ellipse	+ replace blob by fitted ellipse	Same	0.290	0.353	0.343	Discarded: same merged-blob problem
T07 sweep	Threshold sweep: 24 levels; every blob scored by ellipse IoU x solidity x ring contrast; mask NMS; extra eggs if score >= 0.45	At some threshold each egg separates as a clean ellipse	0.369	0.491	0.219	Idea kept; too many extra eggs accepted (instance precision 0.13)
T07b accept0.7	Sweep, accept extras >= 0.7	Fewer false extras	0.436	0.541	0.379	Discarded: 0.85 better
T07b accept0.85	Sweep, accept extras >= 0.85	Fewer false extras	0.501	0.581	0.588	**Kept**: count accuracy 0.13 (T07) to 0.88
T08a watershed	+ grow each core by marker-controlled watershed on the Lab gradient (search = 1.6 x core ellipse)	Recall 0.56: shadowed half of the egg missing; egg interior is smooth so flooding stops at its outline	0.572	0.630	0.658	**Kept** (+0.07, boundary F1 0.40 to 0.51)
T08b grabcut	Instead: GrabCut per egg (GMM colour models + graph cut), same init	Alternative refinement	0.539	0.607	0.614	Discarded: 0.54 < watershed and 1.4x slower
T09a sizeprior	+ log-normal egg-size prior on the candidate score (fit on dev eggs)	Tiny elliptical blobs (earphone bud) beat the egg	0.502	0.576	0.545	Discarded: also penalises partial egg cores, which are smaller than a full egg
T09b refine then score	Grow top-8 cores first, then score grown regions (ellipse x solidity x edge x size)	Score complete regions instead of partial cores	0.363	0.470	0.426	Discarded (0.36). Diagnostic: edge term saturated (egg 10.6, other 4.2, both capped at 3)
T10a calibrated score	Refine-then-score with calibrated score: ellipse^4 x solidity x edge/10 x size x border penalty; extras >= 0.3	Formula chosen offline on a table of 638 refined regions	0.589	0.645	0.682	**Kept** (0.59, count accuracy 0.92)
T10b multiview	+ candidates from 3 views: whiteness, darkness (inverted), smoothness (low local std); top-5 each	Oracle analysis: in 19 of 32 hard images no good core existed (white paper, ash)	0.617	0.655	0.727	**Kept** (0.62, median 0.83)
T11 score v2	Score v2: edge = boundary gradient / outer-ring gradient, + sigmoid brightness contrast; extras >= 0.25	Region table: brightness contrast is the best cue (AUC 0.92); image-median edge fails on textured bg	0.682	0.730	0.784	**Kept** (+0.065)
T12a grow1.3	Watershed search region 1.6 to 1.3 x core ellipse	Large search region lets growth leak	0.703	0.756	0.815	**Kept** (0.703)
T12b grow2.0	Search region 2.0x	Check other direction	0.635	0.687	0.730	Discarded: leaks (0.635)
T12c grow1.15	Search region 1.15x	Check smaller	0.683	0.741	0.805	Discarded: cuts eggs (0.683)
T12d grow1.3 erode9	grow 1.3 + core erosion 9 px	Pull back slightly leaking cores	0.703	0.756	0.815	Discarded: no change, simpler version kept
T12e grow1.3 topk8	grow 1.3 + top-8 candidates per view	Larger candidate pool	0.700	0.754	0.811	Discarded: no gain, 60% slower
T13a hull	+ final mask = convex hull	Smooth jagged watershed outline	0.696	0.752	0.815	Discarded: over-fills eggs partly hidden by fingers/rods
T13b ellipse	+ final mask = fitted ellipse	Same	0.688	0.748	0.805	Discarded: 0.688, boundary F1 drops
T13c smooth11	+ final opening + closing (11 px) per egg	Smooth jagged watershed outline	0.704	0.756	0.815	**Kept = FINAL** (boundary F1 0.584 to 0.590)

Task 3 – Evaluation and analysis

Setup. Ground truth = Task 1 SAM masks (after visual QC); prediction = the frozen classical algorithm (`configs/final_classical.json`, trial T13c). The algorithm was developed on the **dev** split only; the **eval** split (22 images, 9 scene groups never seen during development) was run exactly once. All numbers are reproducible with `scripts/predict_classical.py` + `scripts/task3_analysis.py`.

1. Metrics

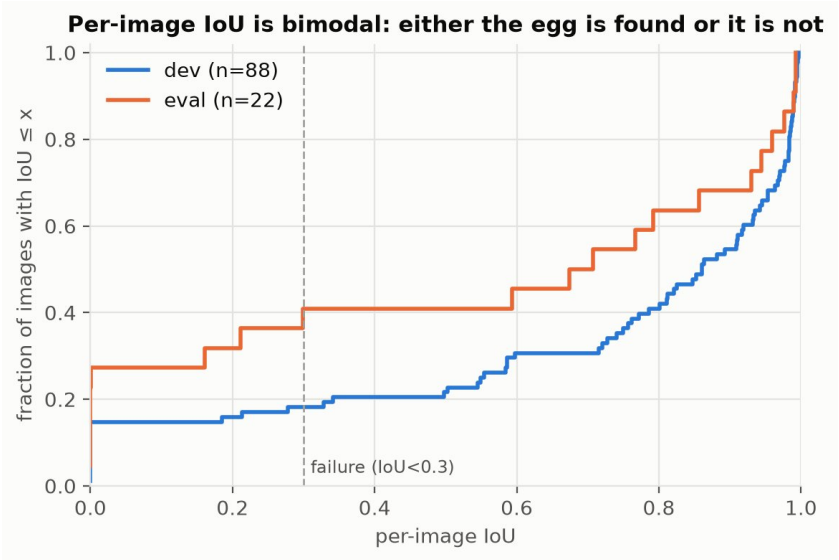
metric	what it measures	dev (88 img / 111 eggs)	eval (22 img / 27 eggs)
mean IoU (95% bootstrap CI)	overlap, averaged over images	**0.704** [0.63, 0.78]	**0.539** [0.36, 0.70]
median IoU	typical image	0.861	0.691
Dice	overlap (F1 of pixels)	0.757	0.592
pixel precision	predicted pixels that are egg	0.794	0.648
pixel recall	egg pixels found	0.754	0.616
boundary F1 (5 px)	outline accuracy	0.590	0.558
instance F1 (IoU ≥ 0.5)	each egg found as its own object	0.815	0.630
egg-count accuracy	correct number of eggs	0.966	1.000
images with IoU ≥ 0.7		69%	50%
images failed (IoU < 0.3)		18%	41%

Why several metrics? IoU/Dice summarise overlap but hide *how* a mask is wrong. Precision vs recall separates "leaks into background" from "misses part of the egg". Boundary F1 checks outline quality, which IoU on a large egg barely notices. Instance metrics and count accuracy matter for Task 4, where eggs must be tracked individually.

Single vs multi-egg images. Multi-egg images are segmented *better* (IoU 0.87 dev / 0.86 eval, instance F1 0.94 / 0.90): they are the marble table and the steel bowl, both with good egg–background contrast, and the joint watershed splits touching eggs correctly. Single-egg images carry all the hard backgrounds (IoU 0.68 / 0.44).

2. The main finding: the result is bimodal

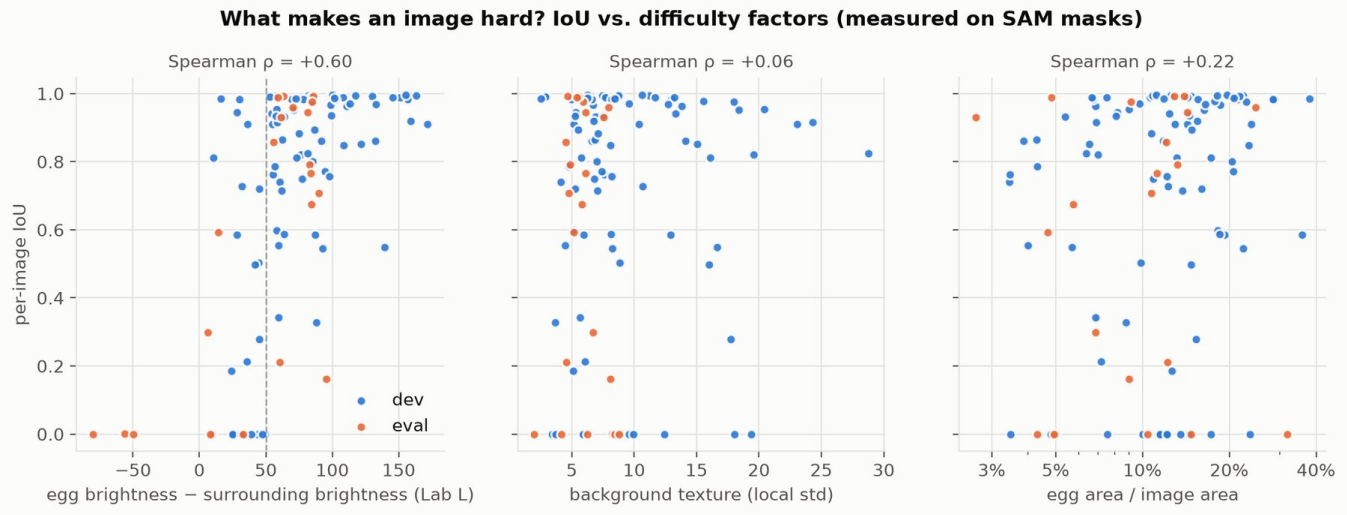
`fig_iou_ecdf.png` – per-image IoU is either high or ~0. When the egg is found (72 images with IoU ≥ 0.7) the outline is close to SAM's: **IoU 0.91, boundary F1 0.79**. The averages are pulled down by images where **the wrong object is selected** (16 dev, 9 eval). So the main gap is **detection** (which region is the egg), not **delineation** (where exactly its border is).



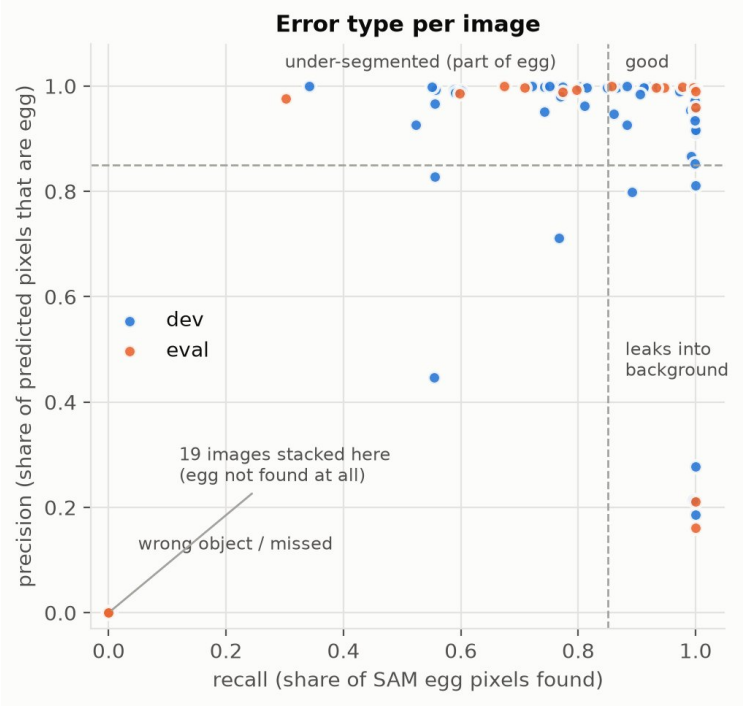
Per-image IoU distribution (ECDF): dev vs eval

3. Why images fail – quantitative

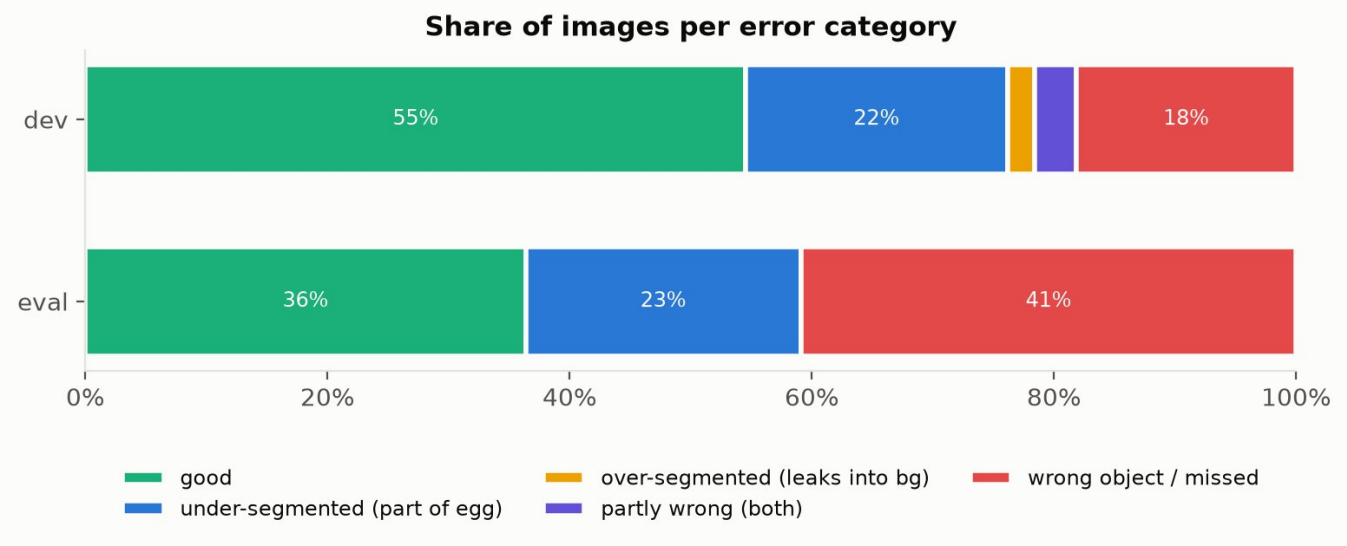
- **Egg–background brightness contrast is the deciding factor** (`fig_difficulty.png`). Contrast = mean Lab-L inside the SAM egg − mean L in a 20 px ring around it. Spearman ρ(contrast, IoU) = **+0.60**. Images with contrast ≥ 50 fail in **3%** of cases (2/76); images with contrast < 50 fail in **68%** (23/34). Median contrast: failures 32, good images 82.
- **Background texture alone does not predict failure** (ρ = +0.06): textured carpets, keyboards and leaves work well when the egg is brighter than them. **Egg size** has only a weak effect (ρ = +0.22).
- **Failures are candidate failures, not scoring failures.** For each dev failure we checked the best region among all refined candidates (`failure_diag_dev.csv`): in **16/16** no candidate had IoU ≥ 0.5 with the egg. A better selection rule would not help; the egg region never forms in any of the three views.
- **Error types** (`fig_precision_recall.png`, `fig_categories.png`): good 55% dev / 36% eval; under-segmented (precision high, recall low) 22% / 23%; wrong object 18% / 41%; leaking into the background is rare (2%): the size prior and the ellipse score stop large leaks.
- **Per scene** (`fig_per_scene.png`): near-perfect on pink tub, patterned cloth, keyboard, printed lid, steel plate (0.96–0.99); worst on dark room (backlit), white sheets, printed plastic bag, egg tray, steel plate on tiles.
- **Dev → eval gap (0.70 → 0.54)**: 10 of the 22 eval images come from four scene types with no counterpart in dev (backlit dark room, white egg tray, white sheet B, red/blue tub lid). The grouped split was built to expose exactly this; a random split would have put near-duplicates of these scenes in dev (28% of eval images would have had a >0.9-similar twin in dev) and reported an optimistic number.



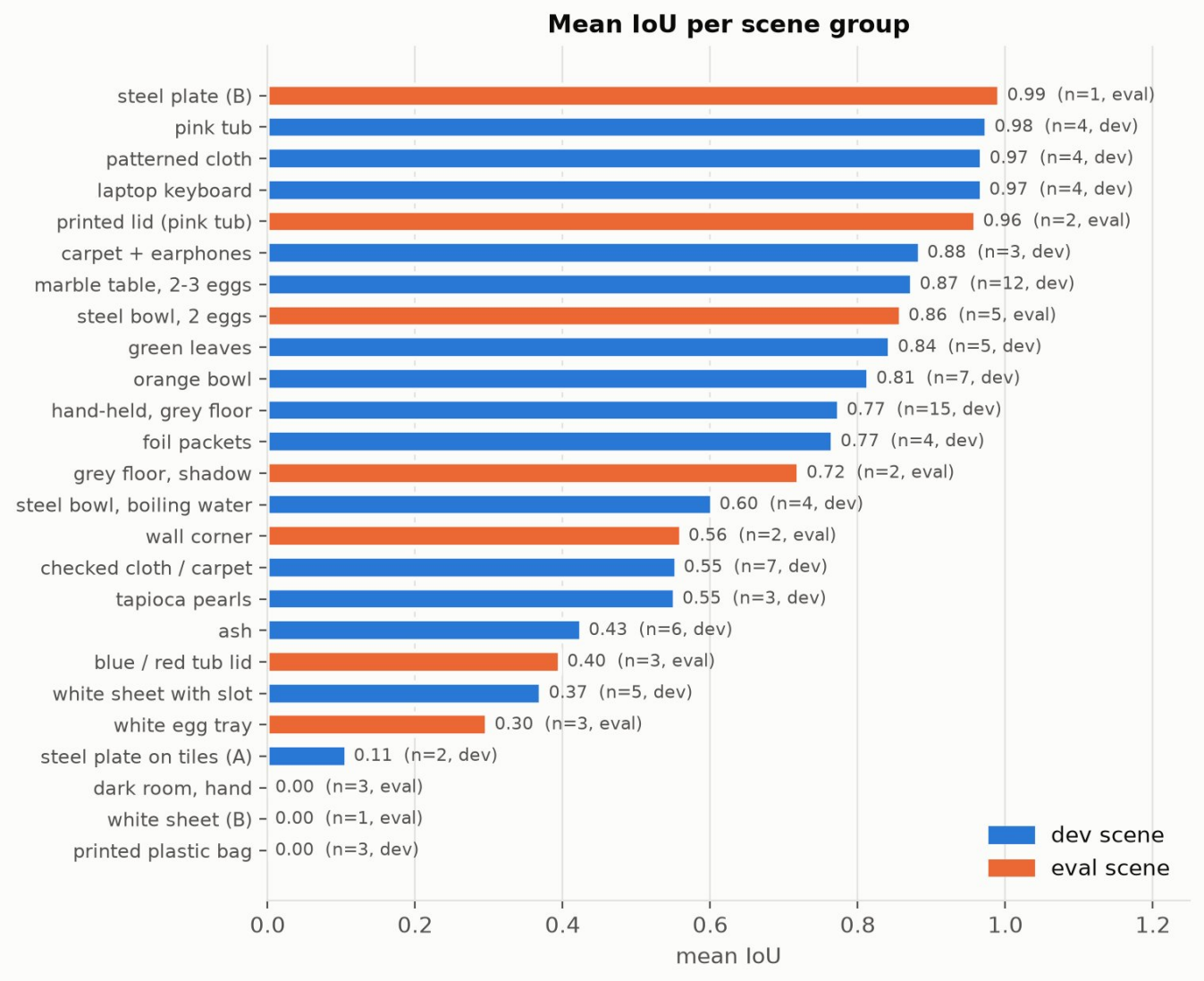
IoU vs. difficulty factors measured on the SAM masks



Error type per image: precision vs recall



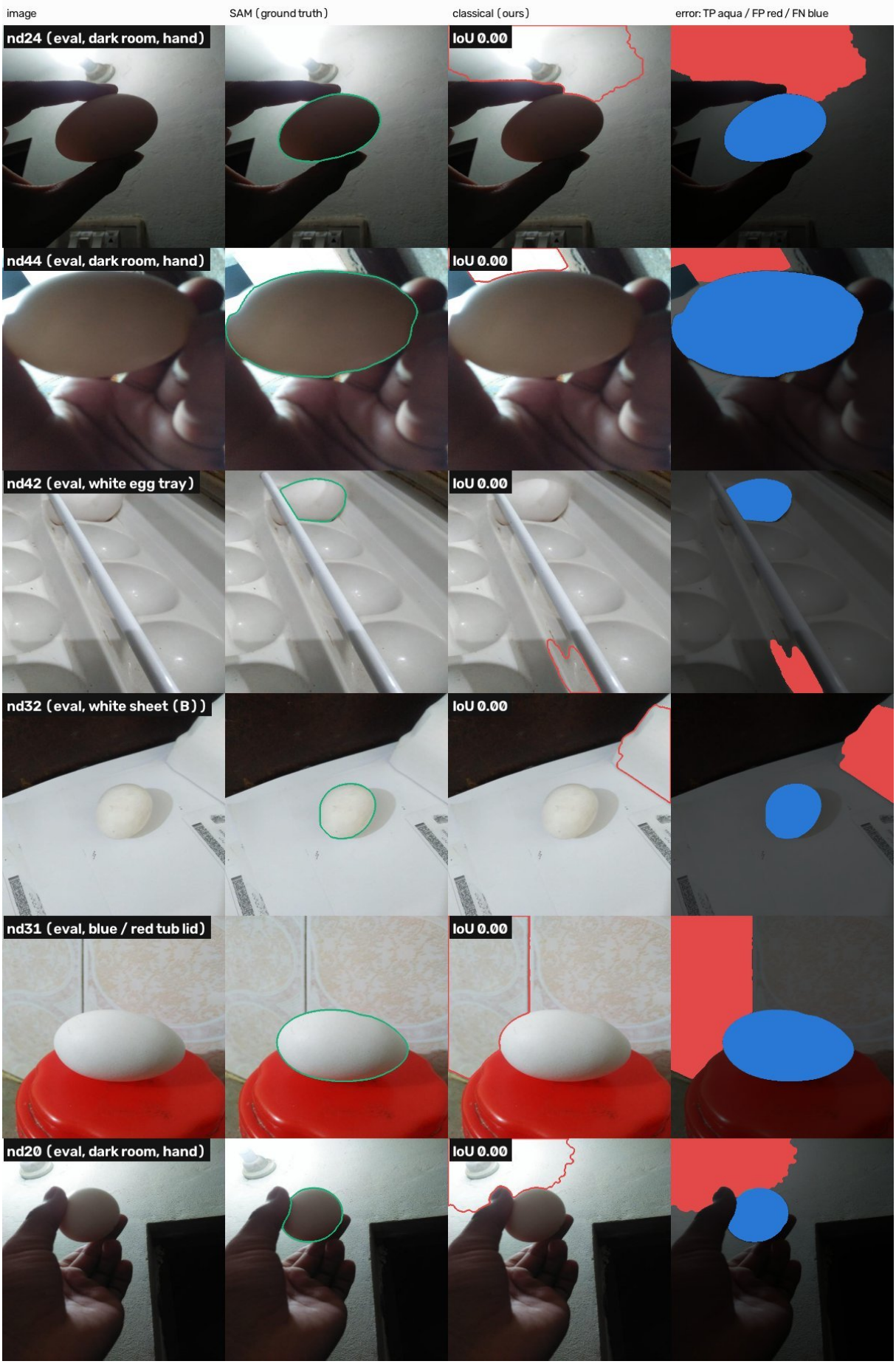
Share of images per error category



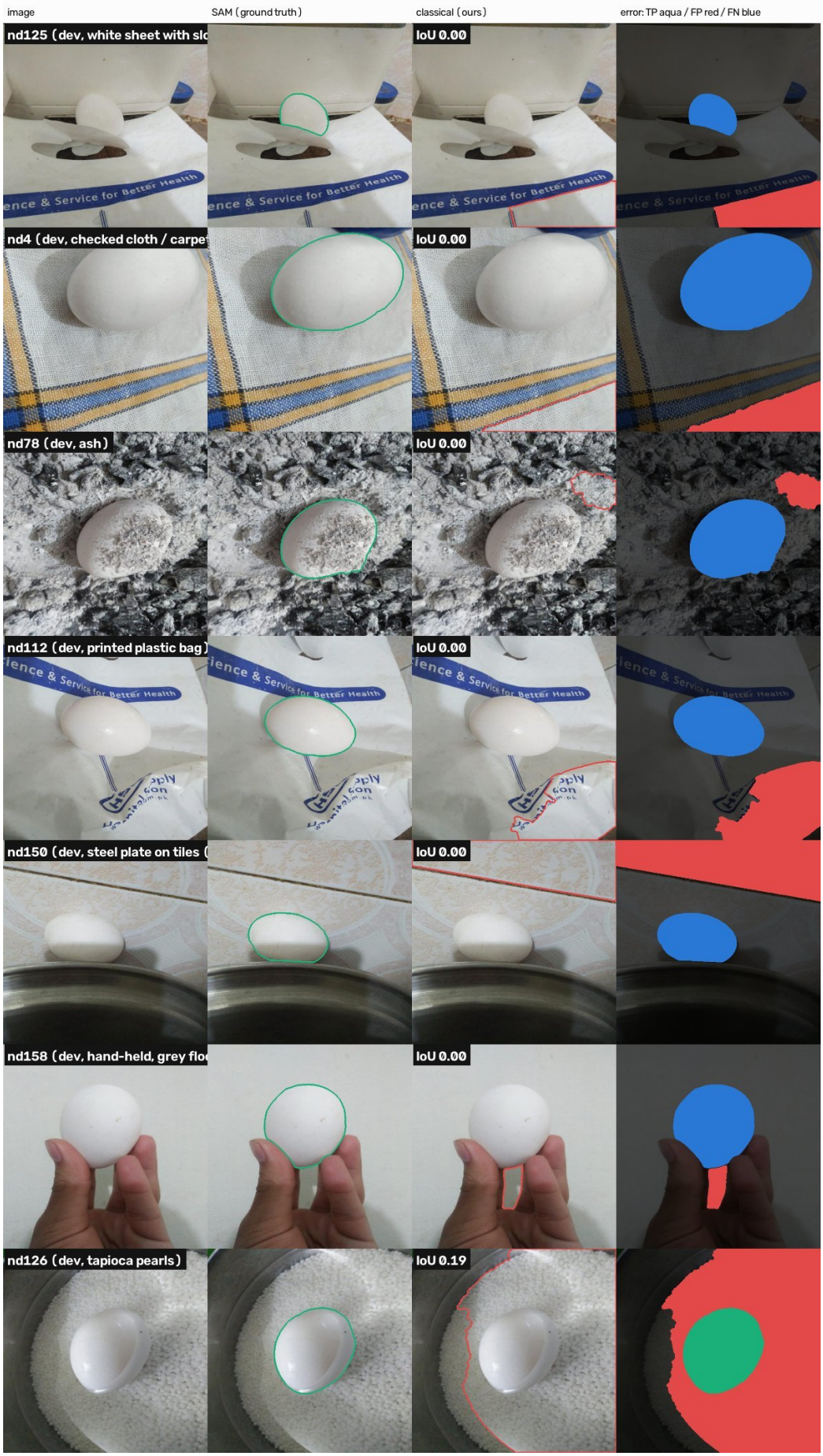
Mean IoU per scene group

4. Why images fail – qualitative (`qual_worst_eval.jpg`, `qual_worst_dev.jpg`, `qual_partial.jpg`)

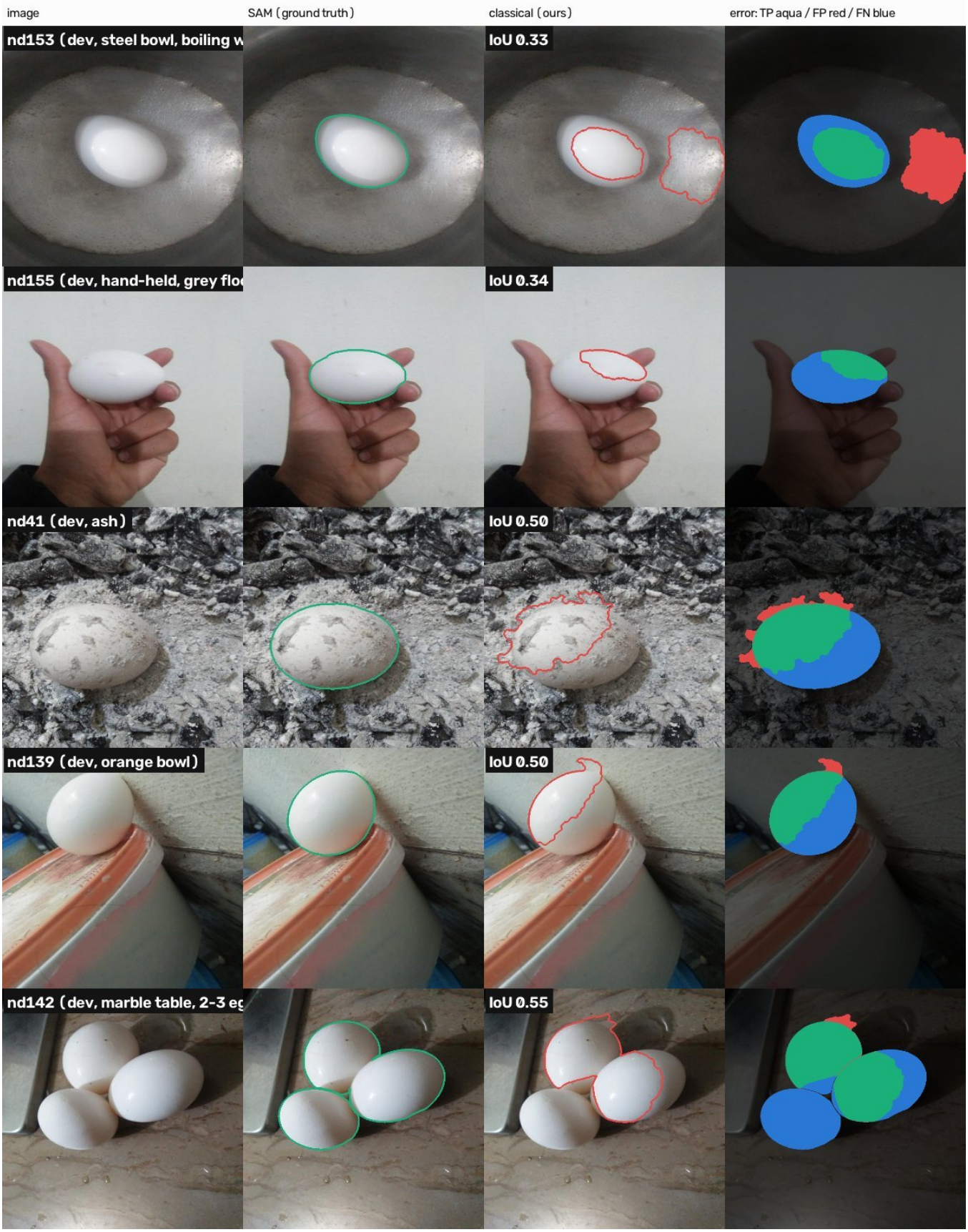
failure mode	examples	what goes wrong
Backlit egg	nd20, nd24, nd44 (dark room)	the egg is *darker* than the lit wall/lamp behind it; every "bright & colourless" cue points at the wall, which is chosen
White egg on white background	nd32, nd42 (tray), nd106/109/112/123/125/143 (sheets, bag)	contrast ≈ 0; the egg never separates in any threshold of any view; a random paper patch, often in an image corner, is chosen instead
Bright, textured background	nd27, nd36, nd78 (ash), nd126 (tapioca)	ash on the shell gives the egg the same brightness *and* texture as the background; whiteness and smoothness views both fail
Shading terminator stops the watershed	nd155, nd139, nd153	the lit side is found, but the edge between lit and shadowed half of the egg is stronger than the egg–background edge, so growth stops inside the egg (under-segmentation)
Specular / glare highlights	nd153, nd135, nd145 (boiling water)	the core is only the highlight; glare blobs on water are bright, round and smooth → extra false eggs
Partial occlusion	nd158 (fingers)	the visible egg part is not an ellipse → low shape score; a finger can win instead
Touching eggs, 3 in a cluster	nd142	two eggs found and split correctly, the third (low contrast to its neighbours) is missed



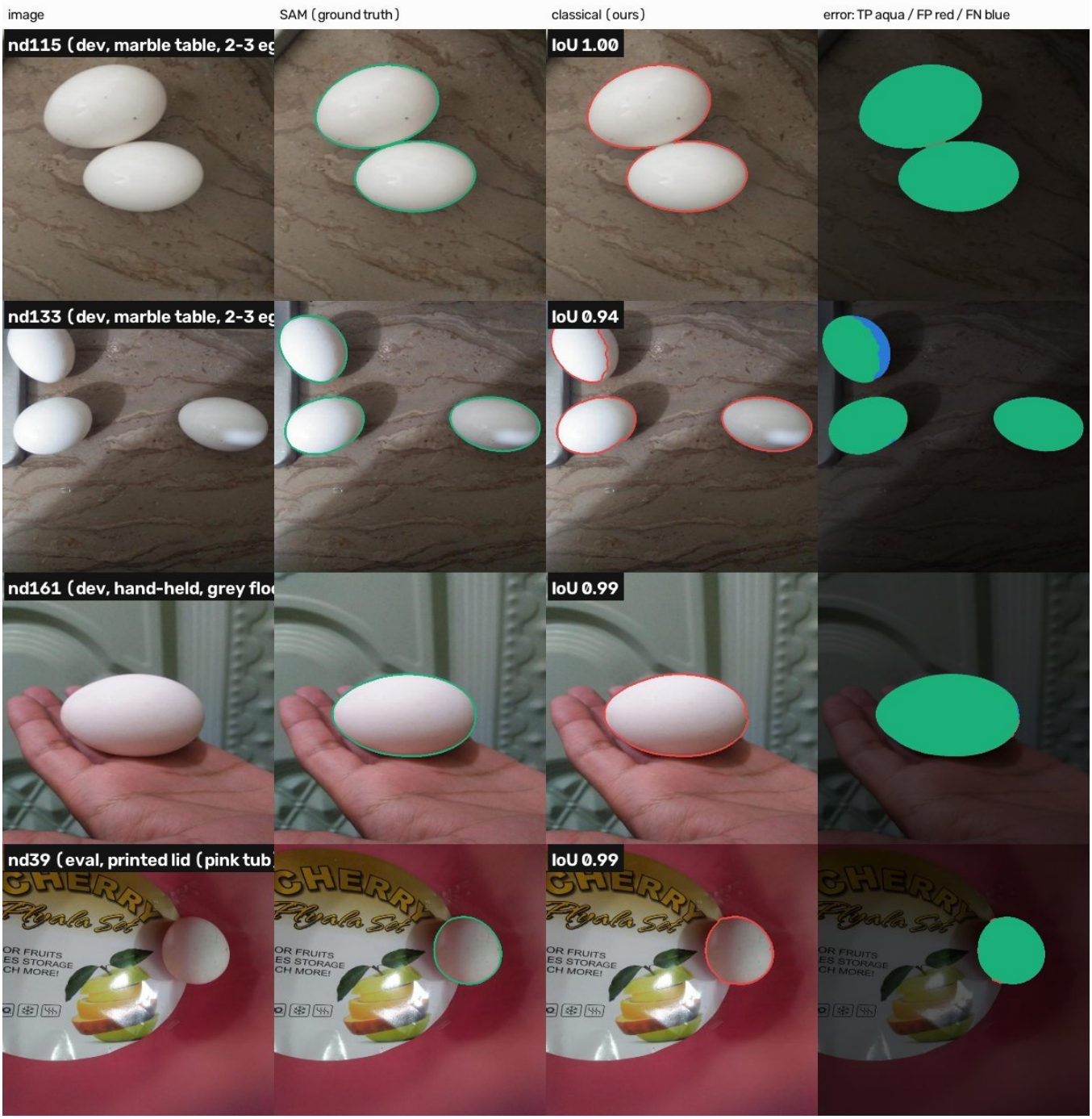
Worst eval images: image, SAM, ours, error map (TP aqua, FP red, FN blue)



Dev failures, one per failing scene type



Partial errors: watershed stops at the shading terminator / glare / shell texture



Successes, including touching eggs split correctly

5. Potential improvements (ordered by expected impact)

1. **Better candidates for low-contrast scenes** – the measured bottleneck. Options: edge-based candidates (Canny + ellipse fitting / Hough ellipse) that do not need the egg to be brighter than the background; local (not global) contrast normalisation (CLAHE) before the threshold sweep.
2. **Backlight handling** – add a fourth view based on *local* rather than global brightness, or detect a backlit image (bright region touching the border + dark foreground) and invert the brightness prior.
3. **Shading-robust growth** – run the watershed on a gradient of the chromaticity / shading-normalised image (e.g. divide by a heavily blurred illumination estimate) so the lit/shadow terminator is weaker than the true outline; or constrain growth with an ellipse fitted to the core boundary pixels that have strong edges.
4. **Reject when unsure** – the wrong regions chosen in failures score 0.08–0.27, often below the 0.25 acceptance threshold used for extra eggs. Applying a minimum score to the first egg as well would turn most wrong detections into "no egg" (same IoU, but no false positives – important for tracking).
5. **Glare suppression** – detect saturated specular blobs (V ≈ 255, tiny chroma) and inpaint/ignore them before computing features.

6. **Learned region scoring (still classical)** – the hand-built score is a product of 6 cues; a logistic regression on the same features, trained on dev with scene-grouped cross-validation, would weigh them optimally. Not done to keep the method fully explainable.
7. **Ground-truth caveat** – SAM masks are pseudo-labels. Visual QC fixed 3/110 images; residual SAM errors (e.g. egg partly hidden by leaves or a rod) set a ceiling on the measured agreement.

Task 4 – Video tracking

Video: "Raw and Boiled Egg Spinning" (YouTube `Y4JyV94y2_A`), 60 s, 1920×1080, 24 fps, 1440 frames. 0–9 s and 42–60 s are title/text cards (no eggs); 9.2–41.7 s show **two white eggs on a dark glossy table** that a hand picks up, carries, and spins.

Pipeline: `video frame → downscale to 640×360 → classical segmenter (Task 2, adapted) → per-frame egg masks → SORT-style tracker → rendered video (1280×720, H.264)`. Run: `python scripts/video_detect.py` (parallel, ~7.5 min on 11 CPU workers) then `python scripts/video_track.py`.

1. Adapting the segmenter (`configs/video.json`)

The Task 2 algorithm was kept unchanged; four settings were added (all *off* by default, so dataset results are reproduced exactly – verified, dev IoU 0.7041). Each was motivated by a failure seen on sample frames:

change	why	evidence
size prior `size_mu` −1.03 → −1.65	dataset eggs are close-ups (median 10% of the image); in this wide shot an egg is ≈2% of the frame	measured egg size in video frames
`min_score = 0.15`	the dataset version always keeps ≥1 egg; title cards must give **no** egg (Task 3, improvement #4)	text "presents" was detected on a title card
`max_chroma = 20`	the colourful "mocomi" logo was detected as an egg	all 138 dataset eggs have mean Lab chroma ≤ 11.3; logo = 34
`min_lcon = 80`	the egg **reflections** in the glossy table are elliptical and were detected (149 frames with 3 detections)	egg-vs-surround brightness contrast: real eggs 114–168, reflections 43–44

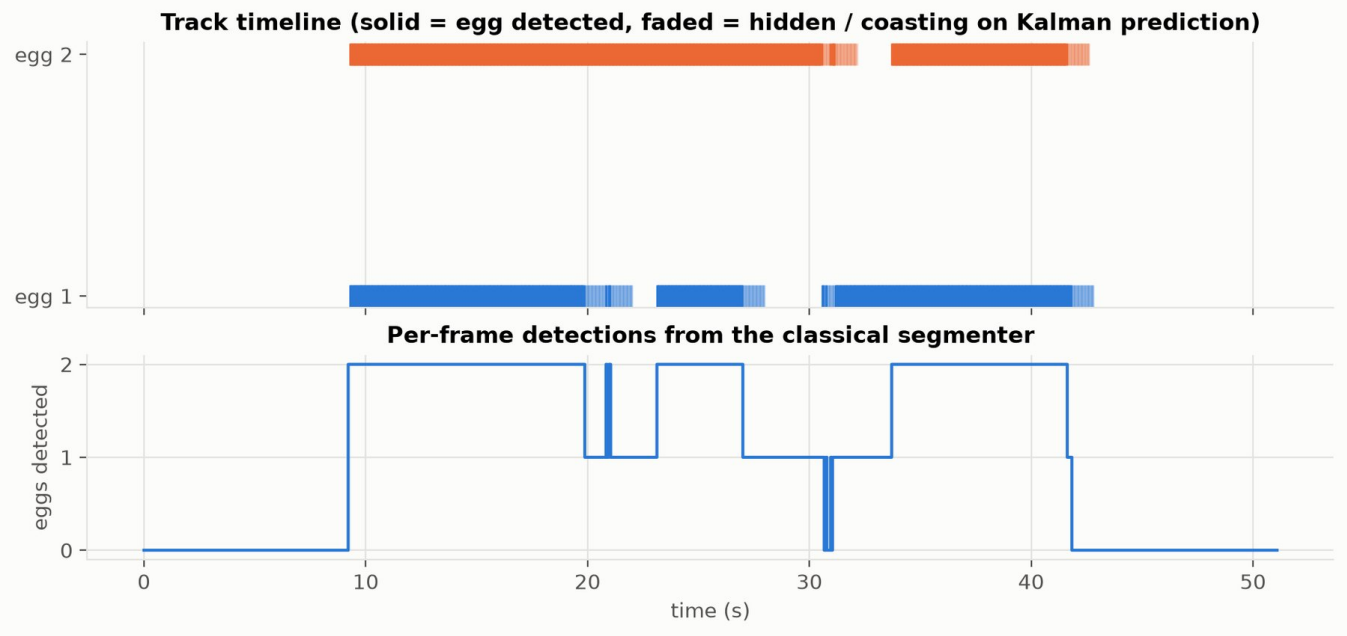
2. Tracker (`src/eggseg/tracking.py`)

A classical SORT-style multi-object tracker:

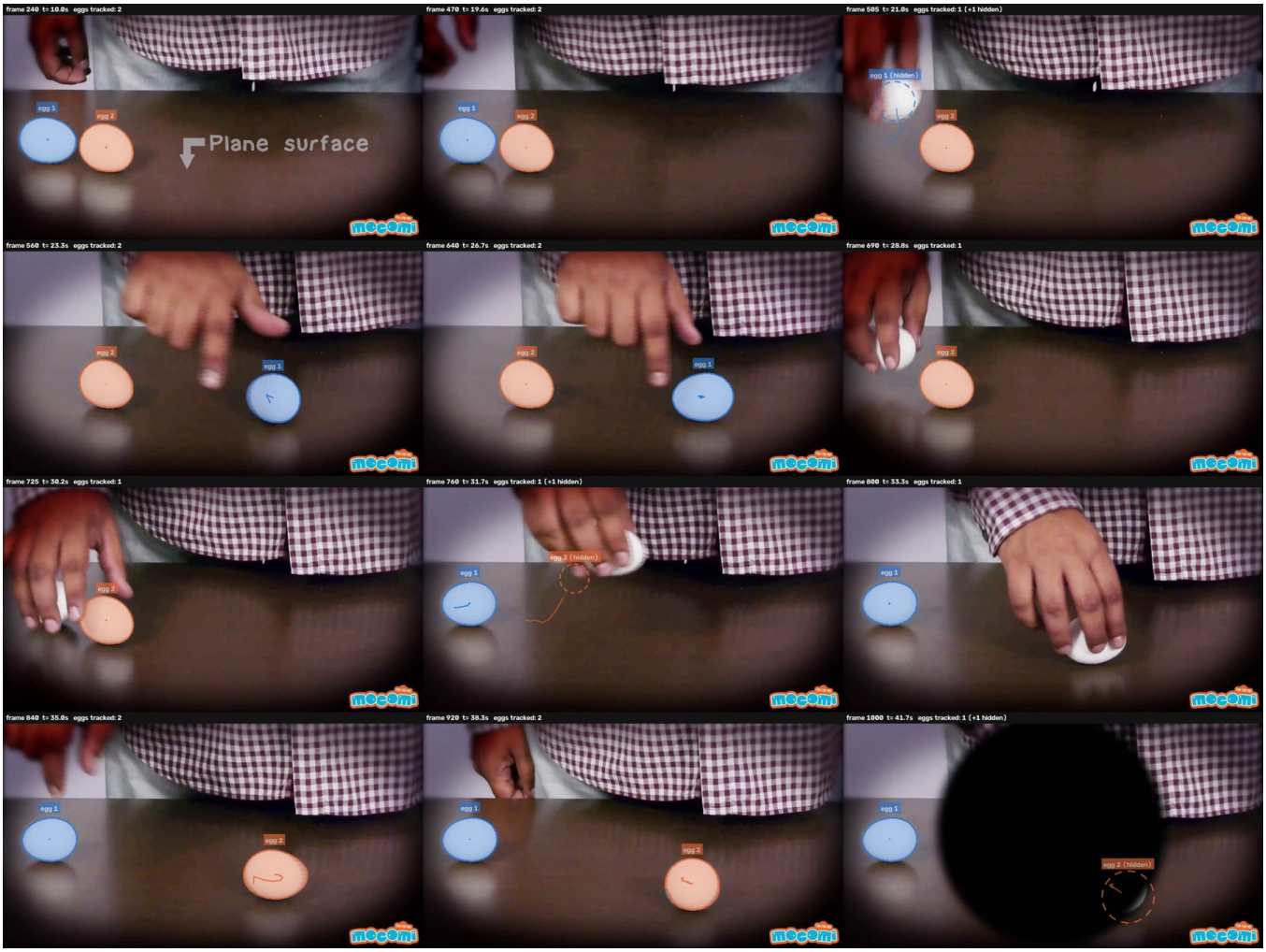
1. **Kalman filter** per egg, constant-velocity state (x, y, vx, vy): predicts the egg centre in the next frame.
2. **Association cost** for each (track, detection) pair: `d + (1 − IoU)`, where d = distance between predicted and detected centre in egg diameters, IoU = overlap of the track's last mask shifted to the prediction with the detection. Pairs with d above a gate (≈1 diameter, growing while a track is unseen) are forbidden.
3. **Hungarian algorithm** (`scipy.optimize.linear_sum_assignment`): globally optimal one-to-one matching.
4. **Track life cycle:** an ID is assigned only after 3 consecutive matches (one-frame false positives never get an ID); unmatched tracks **coast** on the Kalman prediction (shown as a dashed circle "hidden").
5. **Re-identification by elimination.** The eggs look identical, so appearance cannot tell them apart, but the scene logic can: while the other egg is still being tracked, an egg that re-appears must be the missing one. A track unseen for > 1 s may therefore claim a detection anywhere (at a cost always higher than a normal match), and a newly confirmed track inherits the ID of a currently missing egg. Lost tracks are kept for 10 s.

3. Results

	value
IDs created (ground truth: 2 eggs)	**2**
identity errors (12 key frames checked by eye, incl. the hand swapping which egg it holds at 31 s)	**0**
false positives on the 655 egg-free frames (title cards)	**0**
detection coverage in the egg segment (detected egg-frames / 2 × 778 frames)	**84%**
frames with both eggs detected / one / none	540 / 231 / 7
speed (640×360, 1 CPU core)	≈ 3.4 s / frame (parallelised over frames: 7.5 min for the video)



Track timeline and per-frame detections



Rendered output at 12 moments (dashed circle = egg hidden in the hand, coasting on the Kalman prediction)

Iterations (tracking experiment log):

version	IDs created	problem	change
v1 dataset settings	–	logo, title-card text, misses due to size prior	video config (size prior, min_score, max_chroma, min_Icon=15)

version	IDs created	problem	change
v2	7	reflections detected (149 frames with 3 eggs); 2 s coasting too short for an egg carried in the hand	min_Icon 15 → 80
v3	5	no false positives any more, but an egg carried for 3–7 s comes back with a new ID	re-ID by elimination + ID inheritance, max_age 2 s → 10 s
v4 (final)	**2**	–	–

4. Limitations

- **Eggs inside the hand are not detected** (the 16% missing coverage): only fragments between the fingers are visible, which are not ellipses. The tracker bridges these gaps but does not know *where* the egg is while it is carried; the hidden marker is therefore shown for at most 1 s.
- **Re-ID by elimination assumes a fixed number of identical eggs.** With a third egg entering, or two eggs hidden at the same time and re-appearing in swapped positions, IDs could be swapped. An appearance model (colour histogram, size, spots on the shell) or a hand tracker would be needed there.
- **Speed:** 3.4 s per frame is far from real time. Easy gains: run the full detector only every N frames and in between search only near the Kalman prediction (small crop), reuse the previous mask as the watershed marker, and drop the "darkness" view (useless on a dark table).
- Spinning eggs are motion-blurred; the masks become slightly larger/rounder, but are still detected.
- No frame-level ground truth exists for the video; segmentation quality on the video was judged visually. A natural next step is to run the Task 1 SAM notebook on sampled video frames and compute IoU as in Task 3.

Use of AI tools

Claude Code (Anthropic's AI coding assistant, model Claude Opus) was used throughout this project as a pair programmer:

- **Code:** drafting and debugging the Colab notebook, the split script, the classical segmenter, metrics, the experiment runner, diagnostics, the tracker, plotting and report-building scripts.
- **Experiments:** running trials and diagnostics on the dev split and summarising their results.
- **Writing:** first drafts of this report and the README.

How it was used: the work was done step by step. At each step the approach and the underlying concepts (Grounded-SAM, leakage-aware splitting, Otsu, morphology, threshold sweep / MSER idea, marker-controlled watershed, GrabCut, Kalman filter, Hungarian assignment) were discussed before implementation, and design decisions were taken on the basis of the measured results in the experiment log. All results in this report come from the code in the repository and can be reproduced with the commands in the README.